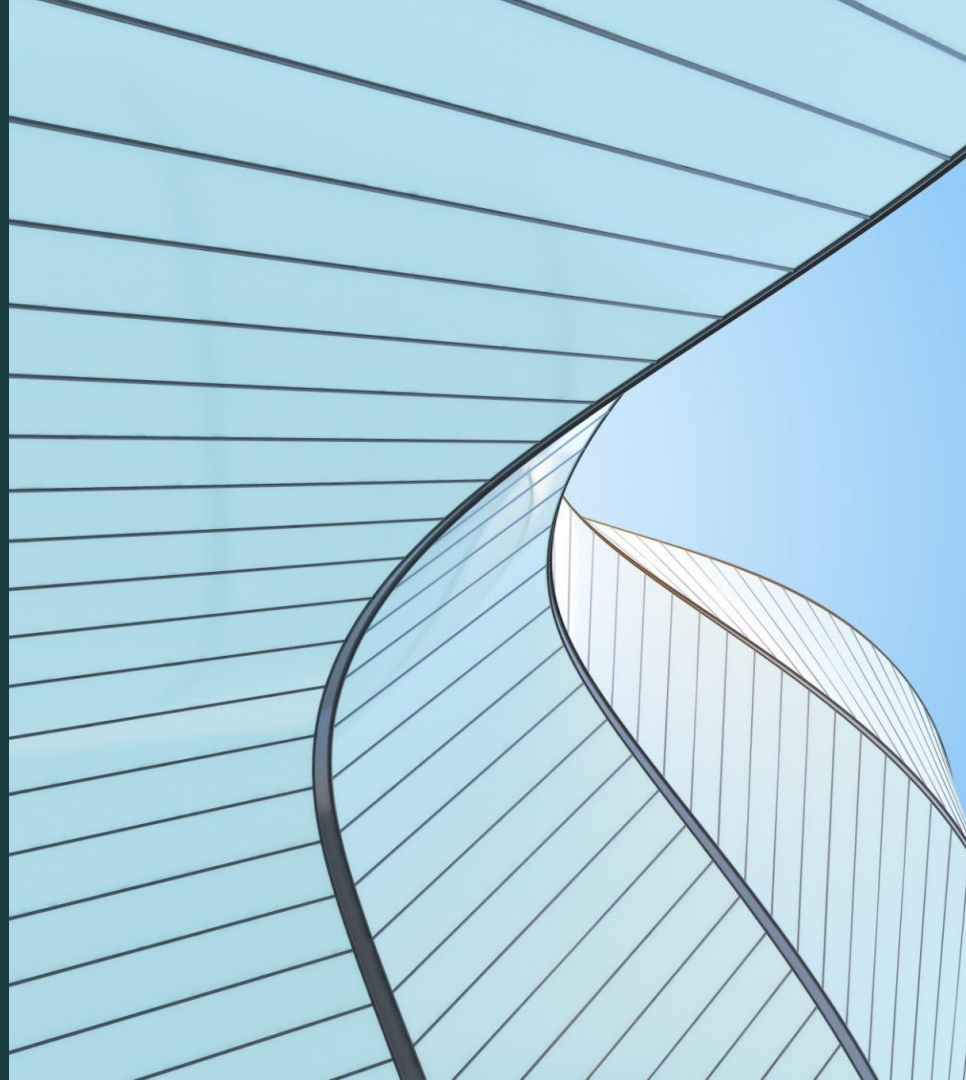


Patrick Guinn
9/25/26

Bank of CLI



Content *overview*

1

Tech Stack

2

ERD

3

OOP Design

4

DAO Pattern

5

Key Decisions

6

Features

Tech Stack

Language: Java 17

Build Tool: Maven

Database: PostgreSQL 18 (via Docker)

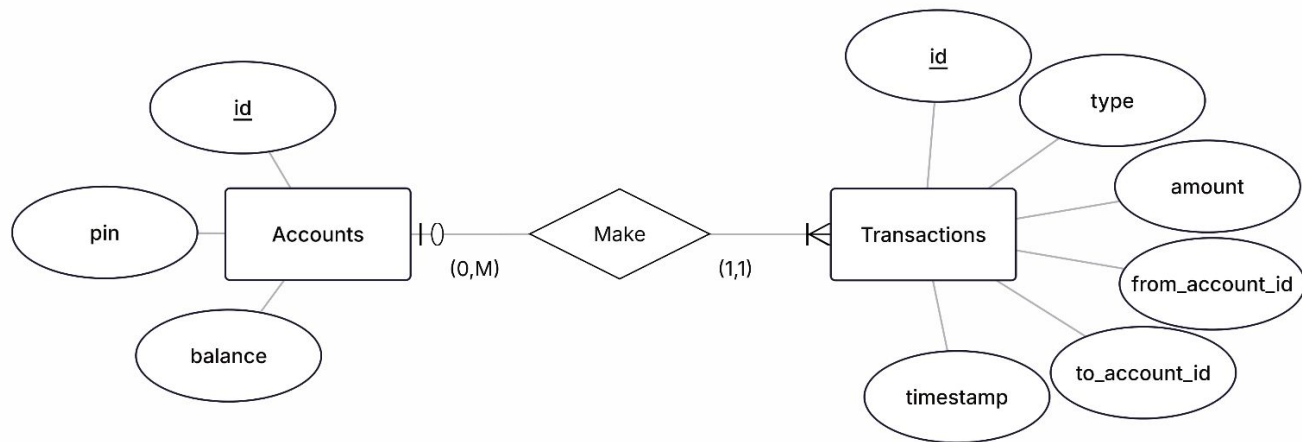
Testing: JUnit 5 + Mockito

Logging: SLF4J + Logback

Version Control: Git & GitHub



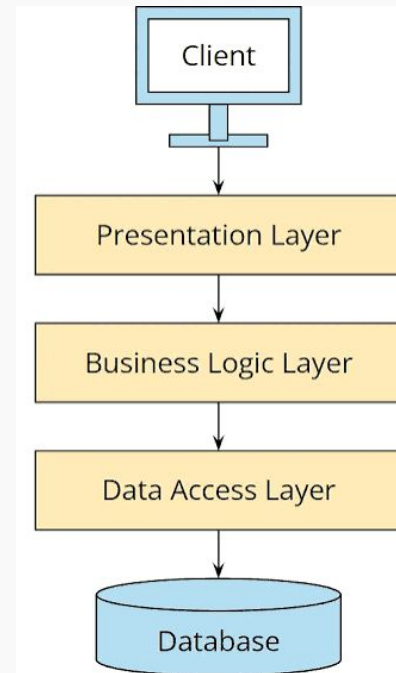
ERD



Three Layer Architecture

```
public class Main {  
    public static void main(String[] args) {  
        AccountService accountService = new AccountService(  
            new AccountRepositoryImpl(),  
            new TransactionRepositoryImpl()  
        );  
  
        ConsoleUI consoleUI = new ConsoleUI(accountService);  
        consoleUI.run();  
    }  
}
```

Layers: API Business Data Access



Feature Explained: Login

ConsoleUI.java

```
private void handleLogin() {
    System.out.print("Enter account ID: ");
    Integer accountId = readAccountId();
    if (accountId == null) return;

    System.out.print("Enter PIN: ");
    String pin = scanner.nextLine().trim();

    try {
        currentAccount = accountService.login(accountId, pin);
        System.out.println("Login successful. Welcome back!");
    } catch (AccountNotFoundException | InvalidPinException e) {
        System.out.println("Login failed: " + e.getMessage());
    } catch (SQLException e) {
        System.out.println("Service unavailable. Please try again later.");
    }
}
```

JDBC

Model

Business

Data
Access

Logging

Custom
Exception

Feature Explained: Login

AccountService.java

```
public Account login(int accountId, String pin) throws SQLException {
    try (Connection conn = DriverManager.getConnection()) {
        return login(conn, accountId, pin);
    } catch (SQLException e) {
        logger.error("Database error during login for account {}: {}", accountId, e.getMessage());
        throw e;
    }
}
```

JDBC

Model

Business

```
Account login(Connection conn, int accountId, String pin) throws SQLException {
    Account account = accountRepository.findById(conn, accountId)
        .orElseThrow(() -> {
            logger.error("Login failed: no account found with id {}", accountId);
            return new AccountNotFoundException("No account found with id " + accountId);
        });
    // Future improvement would include evaluating a hashed pin rather than string
    // evaluation as implemented below
    if (!account.getPin().equals(pin)) {
        logger.error("Login failed: incorrect PIN entered for account {}", accountId);
        throw new InvalidPinException("Incorrect PIN for account " + accountId);
    }

    logger.info("User successfully logged into account {}", accountId);
    return account;
}
```

Data
Access

Logging

Custom
Exception

Feature Explained: Login

AccountRepositoryImpl.java

```
@Override
public Optional<Account> findById(Connection conn, int id) throws SQLException {
    try (PreparedStatement statement = conn.prepareStatement(SQL_FIND_BY_ID)) {
        statement.setInt(1, id);

        try (ResultSet resultSet = statement.executeQuery()) {
            if (resultSet.next()) {
                Account account = new Account (
                    resultSet.getInt("id"),
                    resultSet.getString("pin"),
                    resultSet.getBigDecimal("balance")
                );
                return Optional.of(account);
            }
        }
    }
    return Optional.empty();
}
```

JDBC

Model

Business

Data
Access

Logging

Custom
Exception

Unit Testing and Logging



Demo Time!

Thanks!

Any questions? 